Practical No.10

Aim

To implement Euler's Totient Function $\varphi(n)$ in TypeScript, which calculates the number of positive integers less than n that are relatively prime to n. The implementation includes both naive and efficient methods using prime factorization.

Theory

Euler's Totient Function, denoted as $\varphi(n)$, counts the number of integers k in the range $1 \leq k < n$ for which $\gcd(n,k) = 1$. The function has important applications in number theory and cryptography, particularly in the RSA encryption algorithm.

Key Properties:

- For a prime p: $\varphi(p) = p - 1$
- For distinct primes p, q: $\varphi(p \times q) = (p-1)(q-1)$
- General formula: $\varphi(n) = n \times \prod(1 - 1/p)$ for all distinct prime factors p of n

Code

```
// Euler's Totient Function - TypeScript Implementation
// (n) = number of positive integers less than n that are relatively prime to n

// GCD using Euclidean algorithm
function gcd(a: number, b: number): number {
  while (b !== 0) {
    [a, b] = [b, a % b];
  }
  return Math.abs(a);
}

// Naive totient: count numbers coprime to n
function totientNaive(n: number): number {
  if (n <= 0) return 0;

  let count = 0;
  for (let k = 1; k < n; k++) {
```

```
    if (gcd(n, k) === 1) count++;
  }
  return count;
}

// Efficient totient using prime factorization
// (n) = n × (1 - 1/p) for all distinct prime factors p of n
function totientEfficient(n: number): number {
  if (n <= 0) return 0;

  let result = n;
  let original = n;

  // Check for factor 2
  if (n % 2 === 0) {
    result -= Math.floor(result / 2);
    while (n % 2 === 0) {
      n = Math.floor(n / 2);
    }
  }

  // Check for odd factors
  for (let p = 3; p * p <= n; p += 2) {
    if (n % p === 0) {
      result -= Math.floor(result / p);
      while (n % p === 0) {
        n = Math.floor(n / p);
      }
    }
  }

  // If n is a prime > 2
  if (n > 1) {
    result -= Math.floor(result / n);
  }

  return result;
}
```

```
// Find all numbers less than n that are coprime to n
function findCoprimeNumbers(n: number): number[] {
  const coprimes: number[] = [];
  for (let k = 1; k < n; k++) {
    if (gcd(n, k) === 1) {
      coprimes.push(k);
    }
  }
  return coprimes;
}

// Check if a number is prime
function isPrime(n: number): boolean {
  if (n <= 1) return false;
  if (n <= 3) return true;
  if (n % 2 === 0 || n % 3 === 0) return false;

  for (let i = 5; i * i <= n; i += 6) {
    if (n % i === 0 || n % (i + 2) === 0) return false;
  }
  return true;
}

// Get prime factors of a number
function getPrimeFactors(n: number): number[] {
  const factors: number[] = [];
  let temp = n;

  for (let i = 2; i * i <= n; i++) {
    while (temp % i === 0) {
      factors.push(i);
      temp = Math.floor(temp / i);
    }
  }

  if (temp > 1) {
    factors.push(temp);
  }
```

```
  return factors;
}

// Analyze and display totient function behavior for a number
function analyzeTotient(n: number): void {
  console.log('\n' + '='.repeat(60));
  console.log(`Analysis for n = ${n}`);
  console.log('='.repeat(60));

  console.log(`Is ${n} prime? ${isPrime(n)}`);

  const factors = getPrimeFactors(n);
  if (factors.length > 0) {
    console.log(`Prime factorization: ${factors.join(' × ')}`);
  } else {
    console.log(`Prime factorization: ${n} is prime`);
  }

  const coprimes = findCoprimeNumbers(n);
  const phiNaive = totientNaive(n);
  const phiEfficient = totientEfficient(n);

  console.log(`\nφ(${n}) using naive method: ${phiNaive}`);
  console.log(`φ(${n}) using efficient method: ${phiEfficient}`);
  console.log(`Numbers coprime to ${n}: [${coprimes.join(', ')}]`);
  console.log(`Count: ${coprimes.length}`);
}

// Main function
function main(): void {
  console.log("Euler's Totient Function φ(n)");
  console.log("φ(n) = count of integers k where 1  k < n and gcd(n, k) = 1");

  // Compute for 35 and 37
  [35, 37].forEach(n => analyzeTotient(n));

  // Additional observations
  console.log('\n' + '='.repeat(60));
  console.log('BEHAVIOR ANALYSIS');
```

```
  console.log('='.repeat(60));

  console.log('\nKey Properties Observed:');
  console.log('1. For a prime p: (p) = p - 1');
  console.log('   → Example: (37) = 36 (all numbers 1 to 36 are coprime)');

  console.log('\n2. For distinct primes p, q: (p × q) = (p-1)(q-1)');
  console.log('   → Example: (35) = (5×7) = 4 × 6 = 24');

  console.log('\n3. General formula: (n) = n × (1 - 1/p) for distinct primes p|n');
  console.log('   → (35) = 35 × (1-1/5) × (1-1/7) = 35 × 4/5 × 6/7 = 24');
  console.log('   → (37) = 37 × (1-1/37) = 36');

  console.log('\n4. (n) < n for all n > 1');
  console.log(`   → (35) = 24 < 35`);
  console.log(`   → (37) = 36 < 37`);

  console.log('\n5. (n) is maximized when n is prime');
  console.log('   → For prime p: (p) = p - 1 (maximum possible)');

  // Show range of totient values
  console.log('\n' + '='.repeat(60));
  console.log('Totient Values for n = 1 to 20');
  console.log('='.repeat(60));
  console.log('n  | (n) | Type');
  console.log('-'.repeat(30));

  for (let i = 1; i <= 20; i++) {
    const phi = totientEfficient(i);
    const prime = isPrime(i);
    const type = i === 1 ? '1' : (prime ? 'PRIME' : 'Composite');
    console.log(`${String(i).padStart(2)} | ${String(phi).padStart(4)} | ${type}`);
  }
}

// Run if executed directly
main();
```

Output

Figure 1: Output of Euler's Totient Function Implementation

Conclusion

This practical successfully implements Euler's Totient Function using both naive and efficient approaches. The naive method iterates through all numbers less than n and counts those with $\gcd(n,k) = 1$. The efficient method uses the prime factorization formula $\varphi(n) = n \times \prod(1 - 1/p)$, which is significantly faster for large numbers. The implementation demonstrates key properties including $\varphi(p) = p - 1$ for primes and $\varphi(p \times q) = (p-1)(q-1)$ for products of distinct primes.